

CSE 256 | Lecture 15 | Efficient Adaptation: Parameter-Efficient Fine-Tuning (PEFT)

Ndapa Nakashole

February 24, 2026

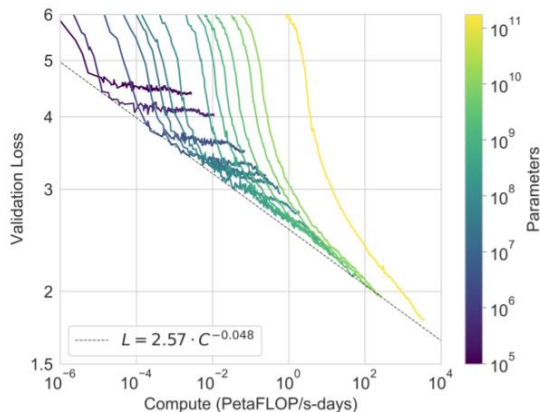
Administrative matters

- ▶ **PA3:** released yesterday, due March 12
- ▶ **OSD Accomodations during Midterm**, two options:
 - ① The Triton Testing Center (TTC) will proctor the exam and provide accommodations as needed (sign up for a slot 3 days in advance)
 - ② Take the midterm in class, with accommodations administered in the classroom.

Recap: Scaling Laws

Simple, predictive “laws” for language model performance as a function of scale (parameters, data, compute)

- ▶ **Old:** tune hyperparameters on big models
- ▶ **New:** tune on small models, extrapolate to large ones



Recap: Post-Training

1) Instruction Fine-tuning; 2) Human preferences from comparison data; 3) Optimize a policy against a reward model using RL

Step 1

Collect demonstration data, and train a supervised policy.

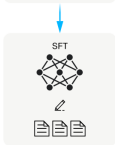
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

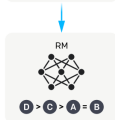
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

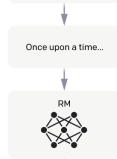


The policy generates an output.

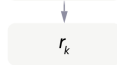


Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Recap: Efficient Adaptation via Prompting

► Fine-tuning a 175B model is:

- ◉ Expensive; Task-specific;
Requires gradient updates

► GPT-3 proposes an alternative: **In-context learning**

- ◉ No parameter updates;
Task specified at inference time; Adaptation through the prompt

► Prompting methods

- ◉ **Zero-shot prompting**: task description only
- ◉ **Few-shot prompting**: task description + examples
- ◉ **Chain-of-thought (CoT)**: task description + examples + reasoning steps/instruct to think step by step
- ◉ **Self-consistency CoT**: sample multiple CoT outputs, take majority vote

```
1 Translate English to French:
2 sea otter => loutre de mer
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => .....
```

task description

examples

prompt

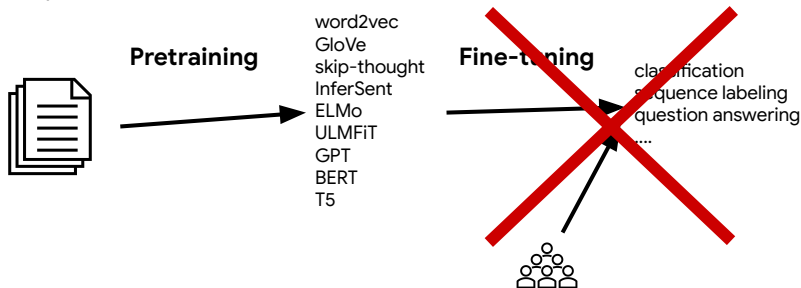
Today

- ① PEFT Overview
- ② Low-Rank Adaptation (LoRA)
- ③ Bias-Only Fine-Tuning (BitFit)
- ④ Adapter Functions
- ⑤ Compression: Knowledge Distillation, Pruning, ...

Parameter Efficient Fine-Tuning

Standard Fine-Tuning Does not Scale

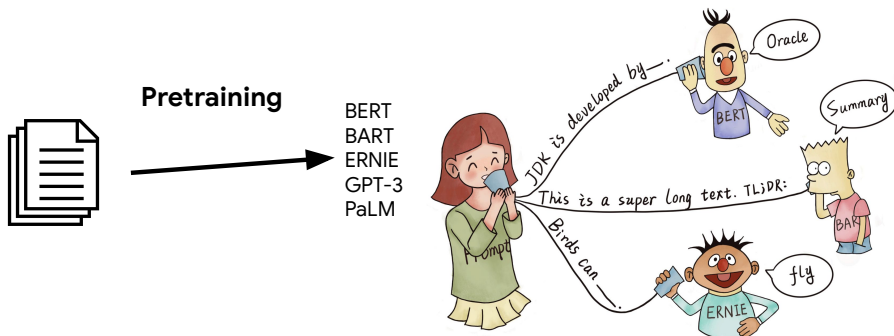
- ▶ With very large models, **full fine-tuning** is **computationally expensive**
- ▶ Maintaining a separate fine-tuned copy per task becomes impractical



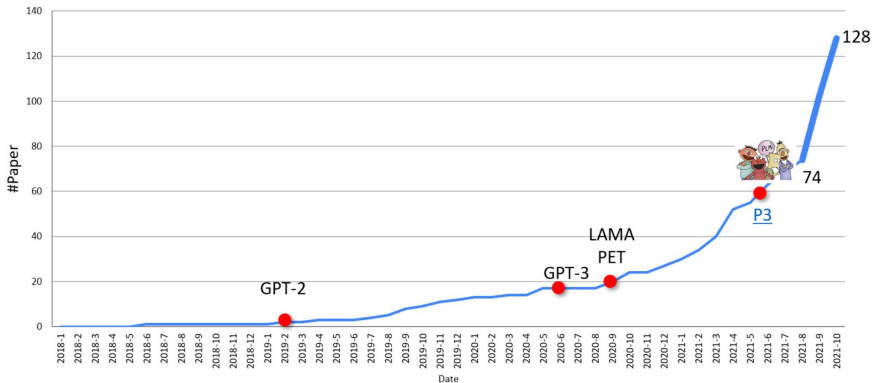
Credit: [NAACL 2019 Transfer learning tutorial](#)

Prompting

- ▶ In-context learning (prompting) enables task adaptation without updating parameters
- ▶ It avoids storing multiple task-specific model copies



Prompt-based Learning Increased Rapidly in NLP



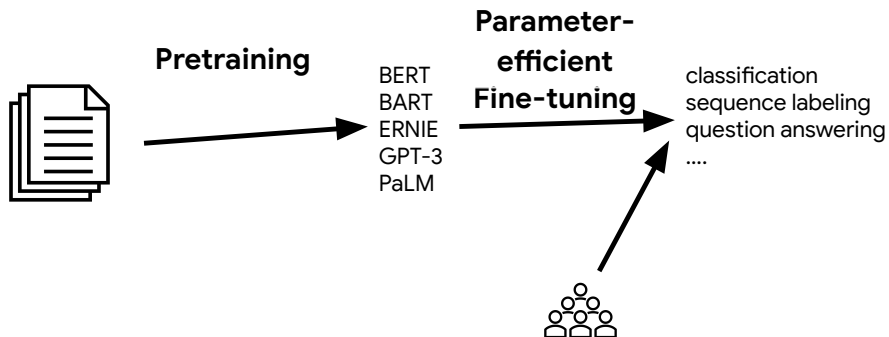
Downsides of Prompt-based Learning

- ▶ **Performance gap:** Prompting often underperforms full fine-tuning
- ▶ **Unclear mechanism:** It is often unclear what information the model extracts from prompts (e.g., random labels can still work) [Min et al., 2022]

Is There a Middle Ground?

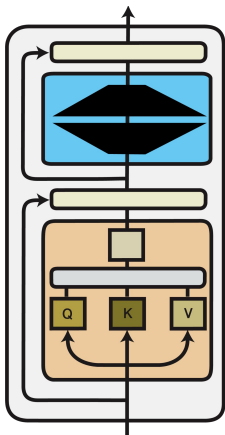
- ▶ Full fine-tuning: High performance, high computational cost
- ▶ Prompting: Lower computational cost, lower reliability
- ▶ Can we adapt large models:
 - ◉ Without updating all parameters?
 - ◉ Without relying entirely on prompts?

From Full Fine-tuning to Parameter-efficient Fine-tuning

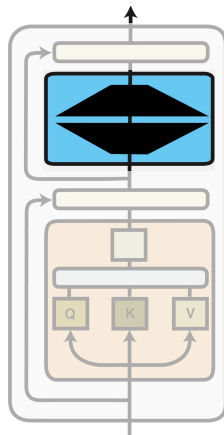


Full Fine-tuning vs. Parameter-efficient Fine-tuning (PEFT)

- **Full Fine-tuning:** Update **all** parameters



- **PEFT:** Update a **small subset** of parameters



Neural Networks, Functions & Modules

- ▶ A neural network can be viewed as a composition of parameterized subfunctions:

$$f_{\theta}(x) = f_{\theta_{\ell}} \circ f_{\theta_{\ell-1}} \circ \dots \circ f_{\theta_1}(x)$$

- ▶ Each subfunction f_{θ_i} has its own parameters θ_i , where $i = 1, \dots, \ell$.
- ▶ $\circ$ denotes the **function composition** operator.

- ▶ A module with parameters ϕ can modify the i -th subfunction via parameter composition:

$$f'_i(x) = f_{\theta_i \oplus \phi}(x)$$

- ▶ In PEFT, we keep θ_i fixed and learn a small set of parameters ϕ .
- ▶ $\oplus$ denotes a **parameter composition** operation (e.g., additive, low-rank, or multiplicative modifications).

Low-Rank Adaptation (LoRA)

Fine-Tuning as Weight Updates

Suppose we fine-tune a pretrained weight matrix:

$$W' = W_0 + \Delta W$$

- ▶ LoRA parameterizes fine-tuning as a **low-rank update**
- ▶ ΔW is constrained to have low rank, i.e., $\text{rank}(\Delta W) \leq r$ where $r \ll d$
 - ◉ LoRA factorizes ΔW as a product of two smaller matrices

$$\Delta W = BA$$

where

$$A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{d \times r}, \quad r \ll d$$

LoRA Update Formulation

- ▶ LoRA parameterizes fine-tuning as a **low-rank update**.

$$W' = W_0 + \Delta W$$

$$\Delta W = BA$$

$$h = W'x = W_0x + BAx$$

$$A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{d \times r}, \quad r \ll d$$

- ▶ Only A and B are trained; W_0 is frozen.
- ▶ Usually applied to **attention projections** (e.g., W_q, W_v).

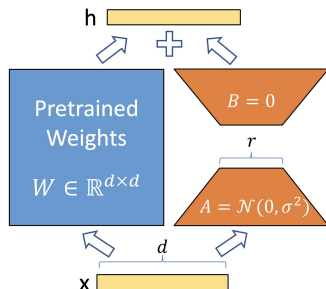
Low-Rank Adaptation (LoRA)

$$h = W_0x + \Delta Wx = W_0x + BAx$$

- Instead of updating $W_0 \in \mathbb{R}^{d \times d}$, we learn a low-rank correction:

$$W' = W_0 + BA$$

- $A \in \mathbb{R}^{r \times d}$ (**down projection**)
- $B \in \mathbb{R}^{d \times r}$ (**up projection**)
- **Initialization:** A is random Gaussian, $B = 0 \Rightarrow \Delta W = 0$ at start of training.



Toy Example: A Rank-1 Matrix

The rank of matrix M is the number of linearly independent columns (or rows).

Consider:

$$M = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Notice:

$$\text{Column 2} = 2 \times \text{Column 1}$$

So the columns are linearly dependent.

$$\text{rank}(M) = 1$$

Even though M is 2×2 .

What Does a Rank-1 Update Actually Do? $r = 1$

Let $\Delta W = BA$ with $A \in \mathbb{R}^{1 \times d}$, $B \in \mathbb{R}^{d \times 1}$

$$A = \begin{bmatrix} a_1 & a_2 & \cdots & a_d \end{bmatrix} \qquad B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_d \end{bmatrix}$$

For any input $x \in \mathbb{R}^d$: $\Delta W x = B(Ax)$

► Ax is a scalar; let $\alpha = Ax$

$$\text{► } \Delta W x = B\alpha = \begin{bmatrix} b_1\alpha \\ b_2\alpha \\ \vdots \\ b_d\alpha \end{bmatrix}$$

Every output is a scalar multiple of B . All outputs lie on a single line in $\mathbb{R}^d$.

More Generally, Rank- r Update ($r \geq 1$)

For any input $x \in \mathbb{R}^d$:

$$\Delta W x = B(Ax)$$

Let

$$\Delta W = BA$$

where

$$A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{d \times r}$$

$$B = \begin{bmatrix} | & & | \\ b_1 & \cdots & b_r \\ | & & | \end{bmatrix}$$

where each $b_i \in \mathbb{R}^d$.

► $Ax \in \mathbb{R}^r$ produces coefficients:

$$Ax = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_r \end{bmatrix}$$

Thus:

$$\Delta W x = B \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_r \end{bmatrix}$$

$\Rightarrow$ The update lives in an r -dimensional subspace of $\mathbb{R}^d$

What Is the Optimal Rank for LoRA?

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Observations:

- ▶ Performance improves quickly from $r = 1$ to $r = 4$.
- ▶ Gains saturate beyond small ranks ($r \approx 8$).
- ▶ Updating more projection matrices often helps more than increasing r .

Why Does LoRA Work? An Empirical Insight

- ▶ Suppose we fully fine-tune a model and obtain: $W_{\text{finetuned}}$
- ▶ The weight update is $\Delta W_{\text{full}} = W_{\text{finetuned}} - W_0$
- ▶ Compute the SVD: $\Delta W_{\text{full}} = U\Sigma V^T$, where:
 - ◉ $U \in \mathbb{R}^{d \times d}$ (left singular vectors)
 - ◉ $V \in \mathbb{R}^{d \times d}$ (right singular vectors)
 - ◉ $\Sigma \in \mathbb{R}^{d \times d}$ diagonal

$$\Sigma = \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \end{bmatrix}$$

$$\text{rank}(\Delta W_{\text{full}}) = \#\{\sigma_i \neq 0\}$$

Empirical observation in many works:

- ▶ The singular values decay rapidly, only a few directions are updated significantly.
- ▶ Full fine-tuning updates are often approximately low-rank - **LoRA exploits this explicitly.**

How Much Does LoRA Reduce Parameters?

Want to fine-tune a weight matrix $W \in \mathbb{R}^{d \times d}$

Full fine-tuning:

- Parameters updated: d^2

LoRA (rank r):

$$\Delta W = BA$$

- $A \in \mathbb{R}^{r \times d} \Rightarrow rd$ parameters
- $B \in \mathbb{R}^{d \times r} \Rightarrow dr$ parameters

Total LoRA parameters = $2dr$

Parameter ratio:

$$\frac{2dr}{d^2} = \frac{2r}{d}$$

$\Rightarrow$ Fraction of parameters trained = $\frac{2r}{d}$

How Much Does LoRA Save? (GPT-3 Example)

GPT-3 hidden size: $d = 12,288$ **LoRA rank:** $r = 16$

Full fine-tuning (one matrix):

$$d^2 = 12,288^2 = 150,994,944 \text{ parameters for one matrix}$$

LoRA update:

$$2dr = 2 \times 12,288 \times 16 = 393,216$$

Fraction trained:

$$\frac{2dr}{d^2} = \frac{2r}{d} = \frac{32}{12,288} \approx 0.26\%$$

$\Rightarrow$ Less than 0.3% of parameters are trained.

Low-rank Adaptation (LoRA)

Exciting Result: LoRA matches or exceeds full fine-tuning while training only a small fraction of parameters.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$.0	94.2 $\pm$.1	88.5 $\pm$ 1.1	60.8 $\pm$.4	93.1 $\pm$.1	90.2 $\pm$.0	71.5 $\pm$ 2.7	89.7 $\pm$.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$.1	94.7 $\pm$.3	88.4 $\pm$.1	62.6 $\pm$.9	93.0 $\pm$.2	90.6 $\pm$.0	75.9 $\pm$ 2.2	90.3 $\pm$.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$.3	95.1 $\pm$.2	89.7 $\pm$.7	63.4 $\pm$ 1.2	93.3 $\pm$.3	90.8 $\pm$.1	86.6 $\pm$.7	91.5 $\pm$.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6 $\pm$.2	96.2 $\pm$.5	90.9 $\pm$ 1.2	68.2 $\pm$ 1.9	94.9 $\pm$.3	91.6 $\pm$.1	87.4 $\pm$ 2.5	92.6 $\pm$.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$.3	96.1 $\pm$.3	90.2 $\pm$.7	68.3 $\pm$ 1.0	94.8 $\pm$.2	91.9 $\pm$.1	83.8 $\pm$ 2.9	92.1 $\pm$.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5 $\pm$.3	96.6 $\pm$.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8 $\pm$.3	91.7 $\pm$.2	80.1 $\pm$ 2.9	91.9 $\pm$.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$.5	96.2 $\pm$.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$.2	92.1 $\pm$.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$.3	96.3 $\pm$.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$.2	91.5 $\pm$.1	72.9 $\pm$ 2.9	91.5 $\pm$.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6 $\pm$.2	96.2 $\pm$.5	90.2 $\pm$ 1.0	68.2 $\pm$ 1.9	94.8 $\pm$.3	91.6 $\pm$.2	85.2 $\pm$ 1.1	92.3 $\pm$.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9 $\pm$.2	96.9 $\pm$.2	92.6 $\pm$.6	72.4 $\pm$ 1.1	96.0 $\pm$.1	92.9 $\pm$.1	94.9 $\pm$.4	93.0 $\pm$.2	91.3

LoRA vs Full Fine-Tuning

Takeaways:

- ▶ Performance is comparable to full fine-tuning.
- ▶ Rank as small as $r = 4$ or $r = 8$ often suffices.

An Even Simpler Idea: Bias-Only Fine-Tuning (BitFit)

Idea: Instead of learning a low-rank update, update only the bias terms $\mathbf{b}$.

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell} \mathbf{x} + \mathbf{b}_q^{m,\ell}$$

$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell} \mathbf{x} + \mathbf{b}_k^{m,\ell}$$

$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell} \mathbf{x} + \mathbf{b}_v^{m,\ell}$$

- ▶ Train only bias parameters. Query and second MLP layer biases matter most.
- ▶ Updates $< 0.1\%$ of parameters
- ▶ Slight performance degradation compared to full fine-tuning

Ways to Constrain Parameter Updates

So far, we've seen two approaches:

- ▶ **Low-rank constraint (LoRA):** Restrict the update to a low-dimensional subspace.
- ▶ **Parameter subgroup constraint (BitFit):** Update only a small subset of parameters (biases).

Another idea: Instead of modifying existing weights, **add small trainable modules.**

Adapters

Adapters (Houlsby et al., 2019)

Idea: Insert small trainable blocks into a pretrained network, while freezing the original weights.

- ▶ The backbone model remains fixed.
- ▶ Each adapter has its own parameters ϕ_i .

Key Difference from LoRA

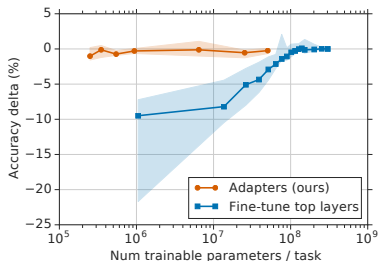
LoRA modifies weight matrices via a low-rank update. Adapters **add new blocks** to the network, which are trained while the original weights are frozen.

Parameter-Efficient Transfer Learning for NLP

Neil Houlsby¹ Andrei Giurgiu^{1*} Stanisław Jastrzebski^{2*} Bruna Morrone¹ Quentin de Laroussilhe¹
Andrea Gesmundo¹ Mona Attariyan¹ Sylvain Gelly¹

Abstract

Fine-tuning large pre-trained models is an effective transfer mechanism in NLP. However, in the presence of many downstream tasks, fine-tuning is parameter inefficient: an entire new model is required for every task. As an alternative, we propose transfer with adapter modules. Adapter modules yield a compact and extensible model; they add only a few trainable parameters per task, and new tasks can be added without revisiting previous ones. The parameters of the original network remain fixed, yielding a high degree of



Adapters in Transformer Models

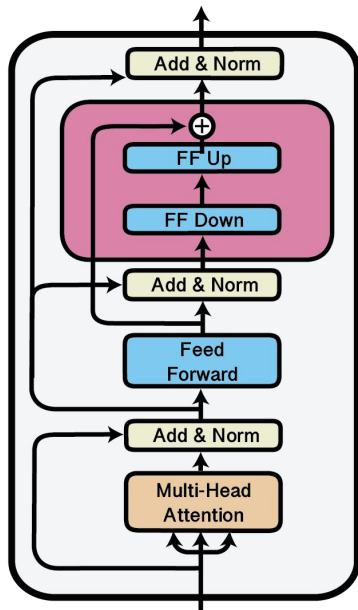
Where are adapters placed?

- Typically inserted:
 - After multi-head attention
 - After feed-forward layers

Bottleneck structure:

$$\text{Adapter}(h) = B \sigma(Ah)$$

- $A \in \mathbb{R}^{r \times d}$ (down projection)
- $B \in \mathbb{R}^{d \times r}$ (up projection)
- $r \ll d$



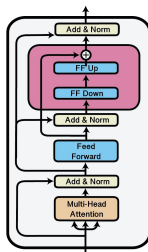
Sequential vs Parallel Adapters

Sequential Adapters

Applied after the layer transformation:

$$h' = \text{Adapter}(f_{\theta}(x))$$

$$= B \sigma(A f_{\theta}(x))$$



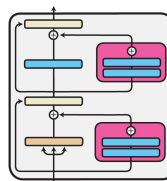
A sequential adapter [Houlsby et

Parallel Adapters

Applied alongside the layer:

$$h' = f_{\theta}(x) + \text{Adapter}(x)$$

$$= f_{\theta}(x) + B \sigma(Ax)$$



Two parallel adapters [Stickland & Murray, 2019]

Sequential vs Parallel Adapters

Adapters can be inserted in two ways.

Sequential Adapters

Inserted after a layer transformation:

$$h' = \text{Adapter}(f_{\theta}(x))$$

- ▶ Replaces the layer output.
- ▶ Modifies the computation path directly.
- ▶ Greater flexibility to alter representations.

Parallel Adapters

Added as a residual correction:

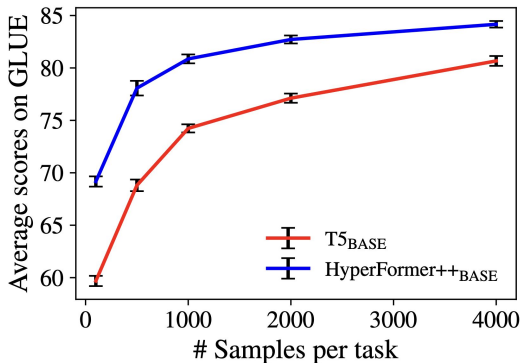
$$h' = f_{\theta}(x) + \text{Adapter}(x)$$

- ▶ Leaves the original layer intact.
- ▶ Adds a small corrective signal.
- ▶ More conservative modification of representations.

Benefits of Adapters: Sample Efficiency

Observation: Adapters often require fewer labeled examples to achieve strong performance.

GLUE results with varying training set sizes (Mahabadi et al., 2021)



Adapters perform competitively even in low-data regimes.

Rescaling-Based Adaptation

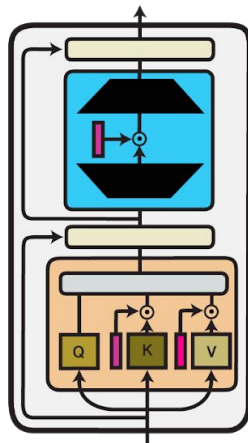
Even simpler than adapters: Instead of adding a function, learn to rescale activations.

$$h' = f_{\theta}(x) \circ \phi$$

- ▶ $\phi \in \mathbb{R}^d$ is a learned vector
- ▶ $\circ$ denotes element-wise multiplication
- ▶ Learns which dimensions to amplify or suppress

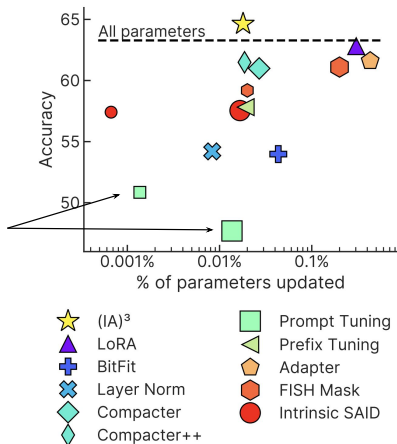
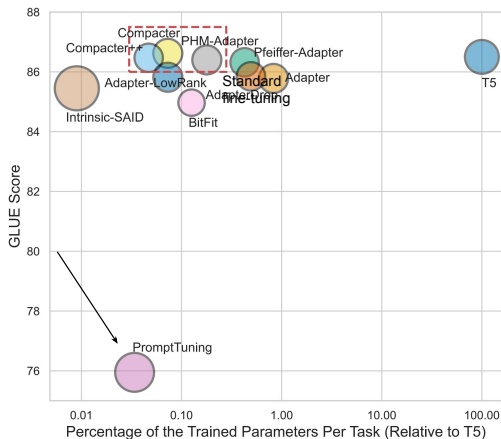
IA³ (Liu et al., 2022):

- ▶ Learns scaling vectors for:
 - ◉ Keys
 - ◉ Values
 - ◉ Feed-forward activations



IA³ [Liu et al., 2022] in the Transformer model

Performance Comparison



Average performance, % of trained parameters per task, and (left) memory footprint (circle size) of different methods on T5-Base (22B) parameters; left) [\[Mahabadi et al., 2021\]](#) and T3-3B (right) [\[Liu et al., 2022\]](#)

TOWARDS A UNIFIED VIEW OF PARAMETER-EFFICIENT TRANSFER LEARNING

Junxian He*

Carnegie Mellon University
junxianh@cs.cmu.edu

Chunting Zhou*

Carnegie Mellon University
chuntinz@cs.cmu.edu

Xuezhe Ma

University of Southern California
xuezhema@isi.edu

Taylor Berg-Kirkpatrick

UC San Diego
tberg@eng.ucsd.edu

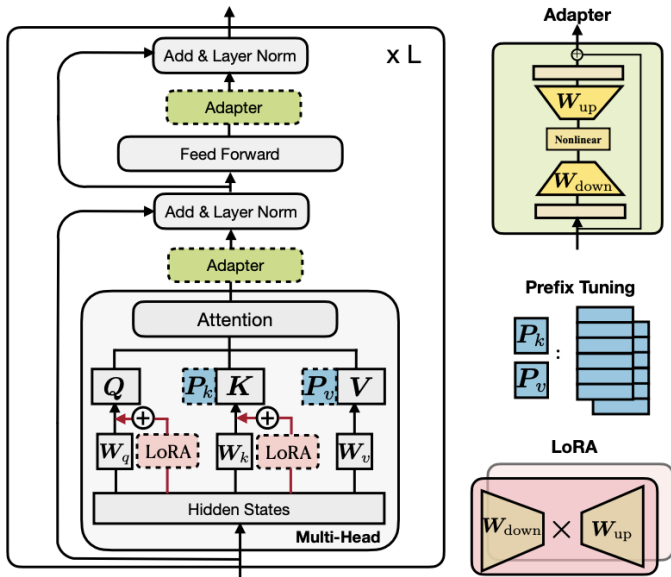
Graham Neubig

Carnegie Mellon University
gneubig@cs.cmu.edu

ABSTRACT

Unifying Computation Functions

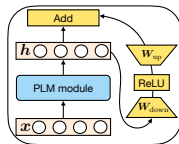
He et al. [2022]



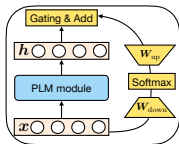
Unifying Computation Functions

He et al. [ICLR 2022]

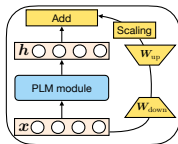
- ▶ He et al. [ICLR 2022] show that LoRA, adapters, etc, can be expressed with a similar functional form
- ▶ Specifically, all methods can be expressed as modifying a model's hidden representation h



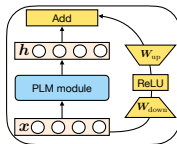
(a) Adapter



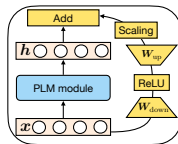
(b) Prefix Tuning



(c) LoRA



(d) Parallel Adapter



(e) Scaled PA

Memory and Space Savings

Consider the memory requirements for training models on: A100 80GB GPU with more than 64GB of CPU RAM.

Model	Full Finetuning	PEFT-LoRA PyTorch
bigsc/TO_3B (3B params)	47.14GB GPU / 2.96 GB CPU	14.4 GB GPU / 2.96 GB CPU
bigsc/mt0-xxl (12B params)	OOM GPU	56 GB GPU / 3GB CPU
bigsc/bloomz-7b1 (7B params)	OOM GPU	32 GB GPU / 3.8GB CPU

Forward Pass Comparison

► Full Fine-Tuning:

- ◉ **All model parameters** participate in the forward computation, contributing to both storage and computation requirements.
- ◉ **Memory Requirement:** All parameters must fit in GPU memory.

► PEFT:

- ◉ **Only small, trainable components** (e.g., adapters) are added to the forward pass.
- ◉ **Memory Requirement:** Only these small components require GPU storage for updates.

Backward Pass Comparison

► **Full Fine-Tuning:**

- ◉ **Gradients** are computed and stored for all model parameters.
- ◉ **Memory Requirement:** High, as every parameter has a corresponding gradient.

► **PEFT:**

- ◉ **Gradients** are computed only for the added trainable components.
- ◉ **Memory Savings:** Significant, as gradient storage is limited to a small subset of parameters.

Optimizer State Comparison

Optimizers stabilize training by keeping info about past gradients

► Full Fine-Tuning:

- ◉ **Optimizer states** (e.g., momentum in SGD, first and second moments in Adam) are required for every parameter.
- ◉ **Memory Cost:** High, as these states must be maintained for the full parameter set.

► PEFT:

- ◉ **Optimizer states** are maintained only for the trainable parameters, drastically reducing memory.
- ◉ **Memory Cost:** Low, with states maintained for only a few matrices.

Model Compression

E.g., knowledge distillation, sparse subnetworks (pruning), quantization, etc.

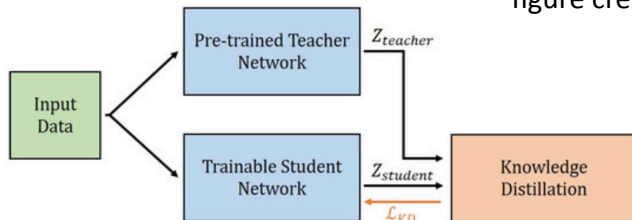
Can be used in conjunction with PEFT (e.g, QLoRA [Dettmers et al., 2023] combines quantization with LoRA)

Knowledge distillation

- ▶ Classic approach from Hinton et al (2015)
 - ◉ Train a large **teacher** model on the original task
 - ◉ Use the teacher to generate "soft labels" (probability distributions) for a dataset
 - ◉ Train a **student** model to match distribution from **teacher**
- ▶ **Intuition:** The teacher's output distribution contains more information than hard labels, e.g., it captures the teacher's uncertainty and inter-class relationships

DistilBERT

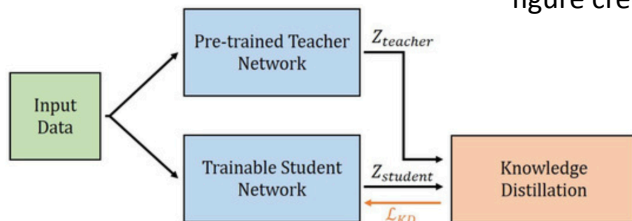
figure credit: Tianjian Li



- ▶ Suppose we have a classification model with output $P_{teacher}(\mathbf{y} | \mathbf{x})$
- ▶ Minimize $KL(P_{teacher} || P_{student})$ to bring student distribution close to teacher
- ▶ Note that this is not using labels - it uses the teacher to "pseudo-label" data, and we label an entire distribution, not just a top-one label

DistilBERT

figure credit: Tianjian Li



- ▶ Use a teacher model as a large neural network, such as BERT
- ▶ Make a small student model that is half the layers of BERT. Initialize with every other layer from the teacher Sanh et al. (2019)

DistilBERT

Model	Score	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Table 2: **DistilBERT yields to comparable performance on downstream tasks.** Comparison on downstream tasks: IMDB (test accuracy) and SQuAD 1.1 (EM/F1 on dev set). D: with a second step of distillation during fine-tuning.

Model	IMDb (acc.)	SQuAD (EM/F1)
BERT-base	93.46	81.2/88.5
DistilBERT	92.82	77.7/85.8
DistilBERT (D)	-	79.1/86.9

Table 3: **DistilBERT is significantly smaller while being constantly faster.** Inference time of a full pass of GLUE task STS-B (sentiment analysis) on CPU with a batch size of 1.

Model	# param. (Millions)	Inf. time (seconds)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

Sparse Subnetworks for PEFT

Sparse Subnetworks for PEFT

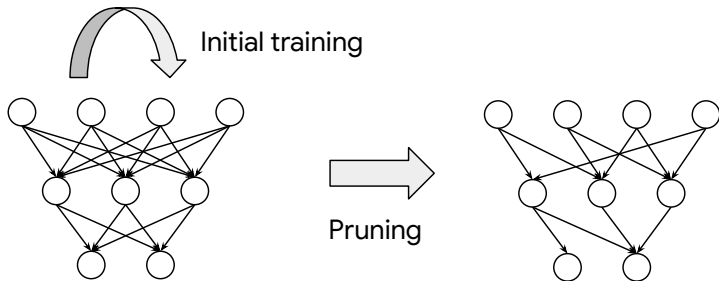
- ▶ Most common sparsity method: **pruning**
- ▶ Pruning can be seen as applying a binary mask

$$\mathbf{b} \in \{0, 1\}^{|\theta|}$$

that selectively keeps or removes each connection in a model and produces a subnetwork

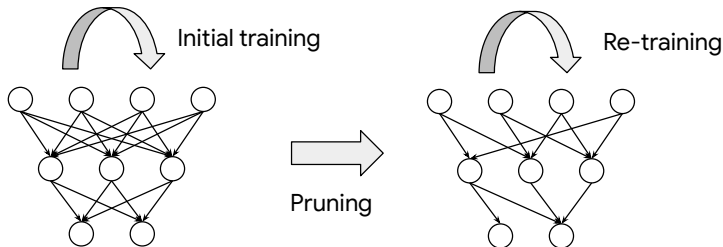
- ▶ Most common pruning criterion: **weight magnitude** [Han et al., 2017]

Pruning



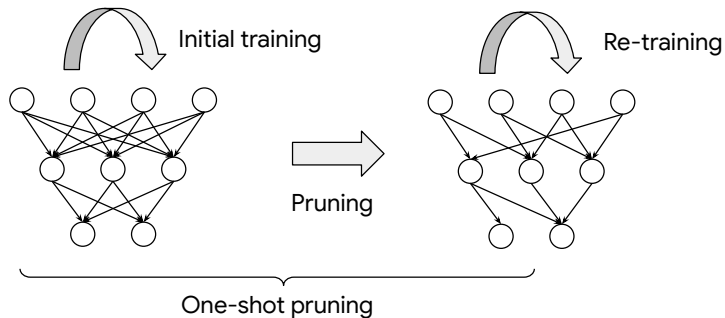
- **Pruning:** a fraction of the **lowest-magnitude weights** are removed

Pruning and Re-training



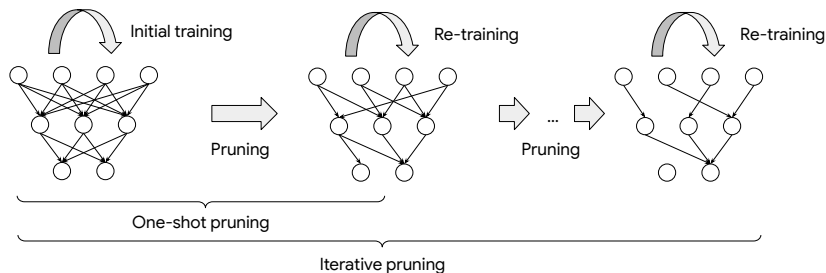
- ▶ **Pruning:** a fraction of the **lowest-magnitude weights** are removed
- ▶ The non-pruned weights are **re-trained**

One Shot Pruning



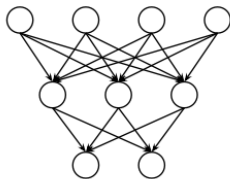
- One-shot pruning: **prune once** and **re-train**

Iterative Pruning



- ▶ Pruning for multiple iterations is more common [Frankle & Carbin, 2019]
 - Provides multiple opportunities to re-evaluate which weights are essential
 - Leads to a more accurate identification of the best subnetworks "winning ticket"

Finding the Right Subnetwork



Starting with a randomly initialized, dense neural network...

- ▶ Is there a subnetwork with the following properties?
 - Same or better results
 - Fewer parameters
 - Shorter training time
 - Trainable from beginning?

THE LOTTERY TICKET HYPOTHESIS: FINDING SPARSE, TRAINABLE NEURAL NETWORKS

Jonathan Frankle

MIT CSAIL

`jfrankle@csail.mit.edu`

Michael Carbin

MIT CSAIL

`mcarbin@csail.mit.edu`

Lottery Ticket Hypothesis

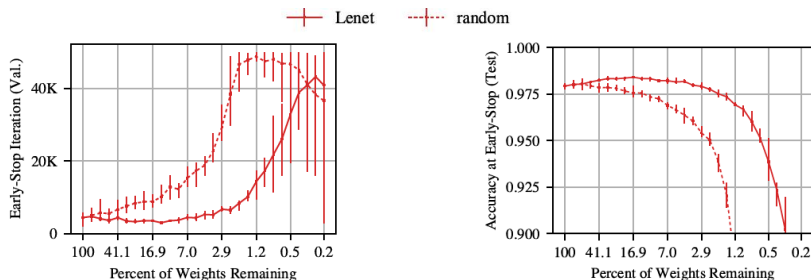
A randomly-initialized, dense neural network **contains a sub-network ("winning ticket")** that is **initialized** such that-when trained in isolation-it **can match the test accuracy of the original network** after training for at most the same number of iterations.

Identifying Winning Tickets

- 1 Randomly initialize a neural network $f(x; \theta_0)$
- 2 Train the network for j iterations, arriving at parameters θ_j
- 3 Prune $p\%$ of the parameters in θ_j , creating a mask m
- 4 **Reset** the remaining parameters to their values in θ_0 , creating the winning ticket $f(x; m \odot \theta_0)$

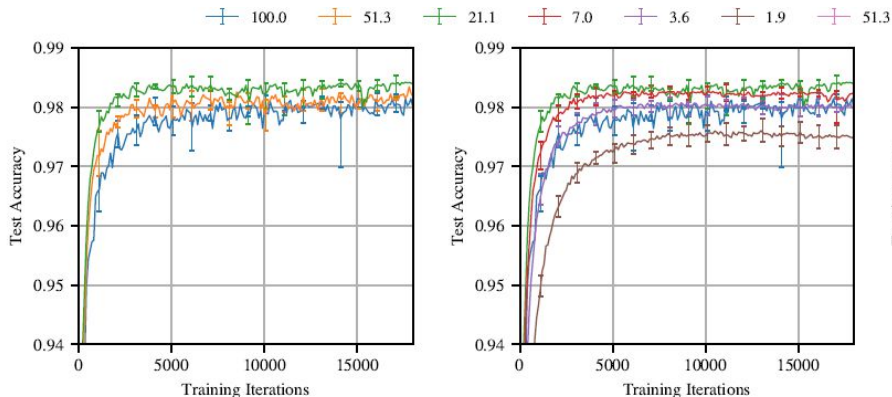
For iterative pruning, steps 2-4 are repeated for n rounds

Winning Tickets Vs Randomly Sampled ones



- ▶ Dashed lines are randomly sampled sparse networks (average of ten trials)
- ▶ Solid lines are winning tickets (average of five trials)

Winning Tickets in Fully-Connected Networks



- Labels: fraction of weights remaining in the network after pruning

Insights from Lottery Ticket Hypothesis

- ▶ The weights that end up in the winning hypothesis **travel much further in the optimization space** compared to those not in the winning hypothesis
- ▶ This suggests that **the good network is not already present in the initialization**
- ▶ Instead, the good network is particularly well-suited to be optimized by SGD.

PEFT: No One-Size-Fits-All Solution

No one-size-fits-all solution.

Choose based on:

- ▶ Compute budget
- ▶ Task complexity
- ▶ Data availability
- ▶ Work is still needed to understand which method works best in which scenarios
- ▶ ...

That's all for today